

PROJECT REVIEW BLUEPRINT

Template for future version reviews of the QPLANT Cryo Leak-Rate Dashboard

Created: 2026-05-12 · Applicable from: v4.0.0+

Usage: Copy this file to REVIEW_vX_Y_Z.md and fill in each section before release.

1 — Version Audit Checklist

1.1 HTML File Inventory

Run this command to list all HTML deliverables:

```
find docs/ -name "*.html" -not -path "**/archive/**" | sort
```

For each file, verify:

File	Expected Version	Actual Version	Canonical?	Stale?	Notes
docs/index.html	vX.Y.Z	check	Hub (auto)		
docs/index_vX_Y.html	vX.Y.Z	check	★ Master		
docs/STAKEHOLDER_PRESENTATION.html	vX.Y.Z	check	★ Executive		
docs/NAVIGATOR.html	vX.Y.Z	check	Navigator		
...					

1.2 Version Consistency Check

```
# Check all HTML files for version references
grep -rn "v[0-9]\+\.[0-9]\+\.[0-9]\+" docs/*.html | grep -v archive
# Check meta tags
grep -rn 'dashboard-version' docs/*.html
# Check VERSION file
cat VERSION
cat VERSION.json | python3 -c "import sys,json; print(json.load(sys.stdin)['version'])"
```

1.3 Canonical vs Derivative Classification

Type	Rule
Canonical	Hand-curated or SSoT-generated, approved for distribution
Auto-generated	Built by scripts from SSoT — rebuild reproduces identical output
Derivative	Copy-and-patch from another file — track source
Archive	Superseded — in docs/archive/ with deprecation banner

2 — Naming Convention Rules

2.1 Versioned Files

docs/index_v{MAJOR}_{MINOR}.html — Master presentation (e.g., index_v4_0.html)
 docs/archive/index_v{M}_{m}_{p}_ARCHIVE.html — Archived versions

2.2 Unversioned Files (always current)

docs/NAVIGATOR.html — Quick-access index (always current)
 docs/STAKEHOLDER_PRESENTATION.html — Executive summary (always current version)
 docs/index.html — Landing hub (auto-generated, shows current version)

2.3 Directories

docs/visualizations_v3/  Charts (name includes generator version, **NOT** data version)
 docs/plots/  Core triage plots (always **current**)
 docs/archive/  **All** superseded files **go** here

2.4 Archive Naming Standard

{original_name}_v{MAJOR}_{MINOR}_{PATCH}_ARCHIVE.{ext}

- Always add deprecation banner in HTML
- Always create/update docs/archive/README.md
- Never edit archived files (breaks traceability)

3 — Idempotency Verification

3.1 Auto-Generated Files

File	Generator	SSoT Input
docs/index.html	src/build_dashboard.py	docs/manifest.json , VER-SION
docs/dashboard.html	src/generate_dashboard.py	data/config.yaml , data/*.json
docs/calculations.html	src/generate_dashboard.py	data/config.yaml
docs/plots/*.html	src/generate_dashboard.py	data/config.yaml , data/scenarios.json
docs/visualizations_v3/*.html	src/generate_visuals_v3.py	data/config.yaml , data/compressor_specs.json

3.2 Rebuild Verification

```
# 1. Record current hashes
find docs/ -name "*.html" -not -path "*/archive/*" -exec sha256sum {} \; | sort > /tmp/before.txt

# 2. Rebuild
python3 src/build_all.py

# 3. Compare
find docs/ -name "*.html" -not -path "*/archive/*" -exec sha256sum {} \; | sort > /tmp/after.txt
diff /tmp/before.txt /tmp/after.txt
```

If diff is empty → build is idempotent 

If diff shows changes → investigate which inputs changed

3.3 SHA256 Manifest Check

```
python3 -c "
import json
m = json.load(open('docs/manifest.json'))
print(f'Version: {m["version"]}')
print(f'Build: {m["build"]["status"]}')
print(f'Tests: {m["tests"]["passed"]}/{m["tests"]["total"]}')
"
```

4 — Navigation Review

4.1 Findability Checklist

- [] Can a new user find the canonical presentation within 2 clicks from `index.html` ?
- [] Does `NAVIGATOR.html` link to ALL current deliverables?
- [] Are stale files clearly separated in `archive/` ?
- [] Is the purpose of each file obvious from its name or surrounding README?
- [] Do all internal links work? (`grep -rn 'href=' docs/*.html | grep -v 'http'`)

4.2 Link Validation

```
# Check for broken internal links
python3 -c "
import re, os
for root, dirs, files in os.walk('docs'):
    for f in files:
        if f.endswith('.html') and 'archive' not in root:
            path = os.path.join(root, f)
            content = open(path).read()
            for m in re.finditer(r'href=\"([^\"]#)+\"', content):
                href = m.group(1)
                if href.startswith('http') or href.startswith('mailto'): continue
                target = os.path.normpath(os.path.join(os.path.dirname(path), href))
                if not os.path.exists(target):
                    print(f'BROKEN: {path} -> {href}')
"
```

5 — Cleanup Decision Matrix

Scenario	Action	Example
File superseded but historically valuable	Archive → <code>docs/archive/</code> with banner	<code>index_v3_1.html</code> → <code>index_v3_1_0_ARCHIVE.html</code>
File superseded and truly obsolete	Delete — no traceability value	Temp build outputs
File still referenced by current version	Keep — update if parameters changed	<code>compressors/HP_Redundancy_Analysis.html</code>
File auto-generated but outdated data	Rebuild from SSoT	<code>dashboard.html</code> after config change
File with ambiguous version	Add version tag or rename	Add <code><meta name="dashboard-version"></code>

Decision Workflow

```

Is the file auto-generated?
YES → Can we rebuild from SSoT?
    YES → Rebuild and verify
    NO  → Archive with note about missing generator
NO → Is it superseded?
    YES → Archive to docs/archive/ with banner
    NO  → Keep, verify version alignment

```

6 — Recursive Build Validation

6.1 Full Build Sequence

```

./setup.sh      # Install deps, verify environment
./build.sh      # Generate all outputs from SSoT
./validate.sh   # Run tests, check consistency
./package.sh    # Create dist/handover.zip

```

6.2 Post-Build Checks

```

# 1. All tests pass
python3 -m pytest tests/ -v

# 2. Manifest is current
cat docs/manifest.json | python3 -m json.tool | head -20

# 3. VERSION matches everywhere
echo "VERSION file: $(cat VERSION)"
echo "VERSION.json: $(python3 -c 'import json; print(json.load(open(\"VERSION.json\"))
[\"version\"]))'"
echo "manifest.json: $(python3 -c 'import json; print(json.load(open(\"docs/mani-
fest.json\")))[\"version\"]))'"

# 4. No stale version references in canonical files
grep -c "v3\." docs/index_v4_0.html docs/STAKEHOLDER_PRESENTATION.html docs/NAVIGAT-
OR.html

# 5. Archive is properly separated
ls docs/archive/

```

6.3 Git Hygiene

```

git status      # No untracked generated files
git diff --stat # Review changes before commit
git log --oneline -5 # Recent history

```

7 — Review Sign-Off

Check	Status	Reviewer	Date
Version consistency	☐		
Canonical files aligned	☐		
Stale files archived	☐		
Navigation working	☐		
Build idempotent	☐		
Tests passing	☐		
Links validated	☐		
Git committed	☐		